

GOLANG

Living Textbook cho Software Engineering và DevOps/SRE

Edition nền móng

Được kiểm chứng với Go 1.27.1 - 20-09-2026

Markdown là nguồn gốc. PDF là bản đọc được, có thể tái tạo cục bộ.

Mục lục của edition này

Đây là edition nền móng. Mục lục tổng thể và thứ tự các phần tiếp theo được giữ trong `book/README.md` để có thể mở rộng mà không giả vờ rằng chúng đã được viết xong.

- Lời nói đầu
- Chương 1. Go toolchain và vòng lặp có thể kiểm chứng
- Chương 2. Một chương trình Go bắt đầu từ package
- Chương 3. Giá trị, kiểu và điều khiển luồng
- Chương 4. Hàm, error và defer: đặt ranh giới trước khi có sự cố

Lời nói đầu

Cuốn sách này dùng để làm gì?

Anh không cần thêm một danh sách cú pháp để chép lại. Điều khó hơn, và đáng học hơn, là tự đi từ một yêu cầu mơ hồ đến một chương trình có thể giải thích, kiểm thử, sửa và vận hành. Go phù hợp với hành trình ấy vì ngôn ngữ nhỏ vừa đủ để ta nhìn rõ phần lớn quyết định, còn thư viện chuẩn lại chạm được vào file, network, process và dịch vụ thật.

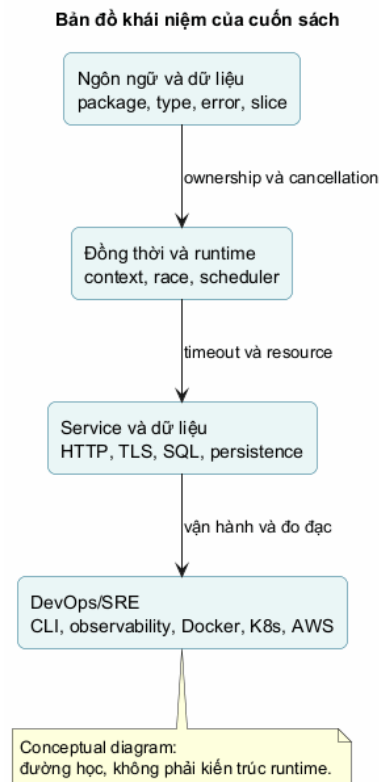
Em sẽ không coi việc đọc xong một chương là bằng chứng anh đã biết. Sau mỗi khái niệm sẽ có lúc phải đoán kết quả, lần theo dữ liệu, đọc code lạ hoặc sửa một lỗi có chủ đích. Nếu bí, hãy ghi giả thuyết trước khi chạy code. Một lần đoán sai nhưng biết vì sao sai có giá trị hơn một lần nhận đáp án đúng từ AI.

Edition này được kiểm chứng với **Go 1.27.1** vào **20-09-2026**. Những phần gắn với release hoặc implementation sẽ được gắn nguồn và mô tả đúng mức của chúng: language specification không phải runtime implementation; một kết quả benchmark trên máy em cũng không phải định luật tự nhiên.

Cách dùng sách

Đọc phần giải thích, gõ lại ví dụ tối thiểu, rồi thay đổi một điều nhỏ trước khi xem lời giải. Hãy để terminal phản biện suy nghĩ của mình. `go fmt`, `go test` và `go vet` không phải nghi thức; chúng là ba câu hỏi khác nhau: code có được trình bày theo chuẩn chung không, hành vi có giữ lời hứa không, và có lỗi tĩnh đáng ngờ không?

Khi đi tới các phần DevOps/SRE, em sẽ luôn quay lại nền tảng. Một timeout HTTP không chỉ là một con số; nó nối với `context`, socket, pool kết nối, retry và ngân sách thời gian của request. Sơ đồ sau là bản đồ khái niệm của đường đi đó, không phải một guarantee của Go runtime.



Quy ước

Code block là code có thể gõ, trừ khi ghi rõ “minh họa”. Output terminal chỉ được nêu khi có thể chạy hoặc khi được gắn nhãn minh họa. Với bài tập, dừng ở đường ngăn trước khi đọc đáp án; đừng biến em thành nút autocomplete có chân.

Nhiệm đọc của edition này

Thành phần	Mục tiêu đọc
Văn xuôi	Hiểu lập luận dài mà không phải nheo mắt trên laptop.
Code	Phân biệt rõ với lời văn, đủ lớn để đối chiếu từng dòng.
Sơ đồ	Làm rõ quan hệ; luôn ghi khi chỉ là conceptual diagram.
Bài tập và đáp án	Buộc dự đoán trước, rồi giải thích nguyên nhân sau.

Chương 1. Go toolchain và vòng lặp có thể kiểm chứng

Go không chỉ là compiler. Khi anh cài Go, thứ xuất hiện trước mặt là lệnh `go`, một cửa vào nhiều công cụ cùng chia sẻ hiểu biết về module và package. Nó có thể chạy chương trình, build binary, format code, chạy test, tải dependency và phân tích một phần lỗi tĩnh. Việc có một command thống nhất làm giảm số mảnh ghép phải nhớ, nhưng không thay thế việc hiểu từng mảnh đang làm gì.

Phiên bản dùng trong edition này là Go 1.27.1. “Mới nhất” không có nghĩa là chạy theo mọi bản thử nghiệm; nó là stable release mà tài liệu chính thức đang phát hành. Mỗi ví dụ có thể dùng ngôn ngữ hiện hành, nhưng project thật vẫn cần chọn go directive theo compatibility policy của chính nó. Từ Go 1.26, `go mod init` có chủ ý tạo directive thấp hơn toolchain một major-minor để khuyến khích tương thích với các bản còn được hỗ trợ. Vì vậy ta sẽ xem `go.mod` là một quyết định tương thích, không phải nhãn trang trí.

Workspace không phải GOPATH cũ

Ngày trước, nhiều Go project bị buộc nằm dưới `GOPATH/src`. Với modules, source có thể nằm ở nơi hợp lý cho project. Một module là đơn vị quản lý dependency; file `go.mod` đặt ranh giới của nó và ghi module path cùng Go version. Package là đơn vị code/import nhỏ hơn: một thư mục thường tạo ra một package, và nhiều package có thể thuộc cùng module.

Sự phân biệt này cứu anh khỏi một lỗi tư duy phổ biến: thấy tên thư mục rồi đoán đó là import path. Import path được xác định từ module path cộng vị trí thư mục; nó không tự sinh từ ổ đĩa. Đó là lý do một project clone từ GitHub có thể build ở máy khác mà không cần cùng tên người dùng Windows.

```

module example.com/opsprobe      <- go.mod xác định gốc module
├─ cmd/opsprobe                  <- package main, tạo executable
├─ internal/check                <- package chỉ module này được import
└─ go.mod

```

`go env GOROOT GOPATH GOMOD` giúp kiểm tra ba điều khác nhau. `GOROOT` là nơi toolchain nằm; `GOPATH` vẫn có ích làm cache và nơi cài một số binary; `GOMOD` cho biết lệnh đang thấy file module nào. Nếu `GOMOD` là `/dev/null` hoặc đường dẫn không như dự kiến, đừng vội xoá cache: kiểm tra thư mục hiện tại trước.

Vòng lặp nhỏ nhưng nghiêm túc

Một thay đổi nhỏ nên đi qua vòng lặp sau. Nó là conceptual diagram, không phải một protocol cứng: với một thử nghiệm một dòng, ta có thể dừng ở bước chạy; với code gửi vào production, thêm test tích hợp và review.



1. Viết một giả thuyết và thay đổi nhỏ nhất có thể kiểm chứng nó.
2. Chạy `go fmt ./...` để bỏ tranh luận vô ích về khoảng trắng.

3. Chạy `go test ./...`; không có file test vẫn là thông tin, không phải thành công giả tạo.
4. Chạy `go vet ./...` khi module đủ hoàn chỉnh để phân tích.
5. Đọc output, sửa nguyên nhân rồi mới lặp lại.

Điểm dễ bị AI che khuất là bước 1. Trước khi hỏi một công cụ, hãy nói được “em tin lỗi nằm ở đâu và dấu hiệu nào sẽ bác bỏ điều đó”. Điều này biến terminal từ máy phát thông báo thành một dụng cụ đo.

Lab: kiểm tra môi trường mà không đoán

Tạo một thư mục rỗng và chạy:

```
go version
go env GOROOT GOPATH GOMOD
go help modules
```

Dừng cố diễn giải tất cả output ngay. Hãy trả lời ba câu: toolchain nào đang chạy, cache/user workspace nào đang được dùng, và thư mục hiện tại đã nằm trong một module chưa. Khi ba câu này rõ, rất nhiều lỗi “Go không tìm thấy package” trở nên có hình dạng cụ thể.

Bài tập

Bạn A đang đứng trong `D:\work\weather` và `go env GOMOD` cho `/dev/null`. Bạn ấy kết luận Go hỏng vì project chưa có `go.mod`. Kết luận đó thiếu điều gì?

ĐÁP ÁN - chỉ đọc sau khi đã tự làm

`/dev/null` thường chỉ cho thấy command hiện tại không thấy module file ở thư mục đang đứng hay thư mục cha. Nó chưa chứng minh Go hỏng. A cần kiểm tra current directory, xác nhận project có thật sự đã `go mod init`, hoặc đi vào đúng thư mục gốc. Chỉ sau đó mới xét tới `GOWORK`, environment override hay toolchain.

Tóm lại

Module là ranh giới dependency; package là ranh giới code/import. Hãy quan sát GOMOD trước khi sửa, và dùng vòng lặp viết - format - test - vet để tạo phản hồi ngắn. Chương sau sẽ đặt một chương trình đầu tiên vào đúng các ranh giới đó, thay vì ném một file `main.go` vào bất kỳ thư mục nào rồi cầu may.

Chương 2. Một chương trình Go bắt đầu từ package

Một file Go không tự nhiên trở thành một chương trình chỉ vì có `func main`. Nó trước hết thuộc về một package. Với executable, package đó phải tên `main` và cần một hàm `main` không tham số, không return. Compiler dùng quy ước này để biết điểm bắt đầu của binary; nó không phải lời mời gọi nhét toàn bộ project vào một file.

Tạo module nhỏ

Trong một thư mục mới, chạy `go mod init example.com/hello`. Module path ở đây là định danh logic; nếu sau này publish code, nó thường trùng địa chỉ repository để người khác import được. Với bài lab cục bộ, `example.com` là domain được dành cho ví dụ, nên không giả vờ rằng ta sở hữu một domain thật.

Tạo `main.go`:

```
package main

import "fmt"

func main() {
    name := "Anh"
    fmt.Printf("Chào %s, hãy thay đổi một dòng rồi chạy lại.\n", name)
}
```

`:=` vừa khai báo biến mới vừa để compiler suy luận kiểu từ biểu thức bên phải. Ở đây `name` có kiểu `string`. Suy luận kiểu không có nghĩa Go là dynamically typed: kiểu vẫn được xác định khi compile, và việc gán sau đó phải phù hợp. Nếu anh viết `name = 42`, compiler dừng trước khi binary tồn tại. Đây là một lỗi rẻ: phát hiện lúc build thường rẻ hơn phát hiện lúc service đang phục vụ người dùng.

Chạy `go run ..` Dấu chấm nghĩa là package trong thư mục hiện tại, chứ không phải “chạy file vừa mở trong editor”. Thói quen này quan trọng ngay

khi package có nhiều file hoặc build tags. `go run build` một executable tạm rồi chạy nó; để tạo artifact có thể đưa cho người khác, dùng `go build -o bin/hello ..`

Đọc lỗi compiler như một tọa độ

Hãy cố ý đổi `import` thành `"fmtt"` rồi chạy lại. Error message có thể trông khó chịu, nhưng nó thường nói ba thứ hữu ích: package hoặc file nào lỗi, dòng nào, và compiler đang thiếu điều gì. Đừng sửa mù theo phần cuối message. Đọc từ dòng đầu liên quan source, sửa lỗi gốc, rồi chạy lại; một lỗi `import` có thể kéo theo nhiều thông báo phụ.

Một lỗi khác hay gặp là `import fmt` rồi không dùng nó. Go coi `unused import` là error vì `import` có thể tạo dependency và tác dụng khởi tạo; giữ `import` chết làm đọc code khó hơn. Đây không phải compiler “khó tính cho vui”, mà là một lựa chọn để dependency nhìn thấy được còn source giữ sạch.

Zero value là một thiết kế tool

Khai báo `var retries int` mà chưa gán cho nó vẫn hợp lệ: giá trị ban đầu của `int` là `0`. `bool` là `false`, `string` là chuỗi rỗng, `pointer/map/slice/channel/ function/interface` là `nil`. Zero value giúp type có thể dùng được ngay mà không cần constructor khi điều đó hợp lý, nhưng không biến `nil` thành an toàn. Việc ghi vào `nil map` sẽ panic; đọc `nil map` thì hợp lệ và trả zero value của element. Ta sẽ quay lại chi tiết aliasing và `nil` ở chương dữ liệu.

```
var attempts int
var ready bool

fmt.Println(attempts, ready) // 0 false
```

Đoạn trên in output xác định. Nếu ví dụ sau này phụ thuộc thời gian, `network` hoặc `scheduler`, sách sẽ nói rõ điều đó thay vì hứa hẹn một output đẹp nhưng không lặp lại được.

Điểm nối sang DevOps/SRE

Một CLI health check nhỏ hay exporter lớn đều bắt đầu từ package main, nhưng đa số logic không nên sống ở đó. main thường chỉ wiring: đọc config, tạo dependency, gọi luồng chính và quyết định exit code. Logic kiểm tra HTTP, parse file hay tính retry nên ở package riêng để test không cần fork process. Chưa cần xây kiến trúc ngay hôm nay; chỉ cần thấy rằng cách ta khởi động chương trình đã ảnh hưởng trực tiếp tới cách ta kiểm thử và vận hành nó sau này.

Bài tập

Không chạy code trước. Dự đoán: chương trình dưới đây compile hay không? Nếu compile, nó in gì? Sau đó hãy tạo module và kiểm tra dự đoán.

```
package main

import "fmt"

func main() {
    var port int
    port = port + 8080
    fmt.Println(port)
}
```

ĐÁP ÁN - chỉ đọc sau khi đã tự làm

Nó compile và in 8080. port nhận zero value 0, rồi phép cộng tạo giá trị mới để gán lại. Cạm bẫy là nhầm zero value với “chưa có giá trị” trong mọi miền nghiệp vụ. Với port, 0 còn có nghĩa đặc biệt trong một số API network; vì vậy khi dữ liệu cần phân biệt “không được cấu hình” với số 0, ta cần một biểu diễn khác như pointer, boolean riêng hoặc type có ý nghĩa rõ ràng.

Tóm lại

`package main` và `func main` là entry point theo quy ước của executable. `go run` . chạy package hiện tại; `go build` tạo artifact. Type inference vẫn là static typing, còn zero value là một tính năng thiết kế có ích nhưng cần được đọc trong ngữ cảnh. Chương tiếp theo sẽ dùng các kiểu và control flow để biến một chương trình chỉ in text thành một chương trình có quyết định rõ ràng.

Chương 3. Giá trị, kiểu và điều khiển luồng

Sau khi chương trình đã có điểm bắt đầu, câu hỏi kế tiếp không phải là Go có bao nhiêu keyword mà là dữ liệu này có ý nghĩa gì, và chương trình được phép làm gì với nó. Kiểu trả lời phần đầu; control flow trả lời phần sau. Nếu hai thứ mơ hồ, một tool vận hành nhỏ cũng nhanh chóng thành chuỗi if mà không ai dám sửa.

Kiểu là ràng buộc có ích

Go có suy luận kiểu ở chỗ dễ thấy nhất: `attempts := 3` tạo biến `attempts` có kiểu `int`. Nhưng compiler vẫn phải biết kiểu chính xác trước khi chạy. Vì vậy đoạn sau không hợp lệ; Go không tự đổi string sang số chỉ vì người đọc đoán được ý định.

```
var timeoutMS int = 500
timeoutMS = "500" // compile error
```

Sự từ chối này hơi bất tiện vào ngày đầu, nhưng rất có ích với cấu hình và dữ liệu từ network. Một field "500" trong JSON, một duration 500ms, và một con số 500 có thể cùng trông giống nhau nhưng là ba biểu diễn khác nhau. Chọn đúng kiểu sớm sẽ để compiler bắt những nhầm lẫn rẻ nhất.

Khi ý nghĩa không rõ từ biểu thức bên phải, hãy viết kiểu. `var retries uint8` không chỉ thông báo phạm vi số; nó còn buộc người đọc hỏi liệu giới hạn 255 có phải quy tắc nghiệp vụ thật hay không. Với port, latency hay số byte, `int` là điểm bắt đầu thường tiện, nhưng API public nên thể hiện đơn vị trong tên: `timeoutMS`, `maxBytes`, `attempts`. Go không có type an toàn đơn vị dựng sẵn; tên tốt và boundary validation vẫn rất quan trọng.

Hằng số, conversion và phép toán

`const` biểu diễn một giá trị không được gán lại trong source. Hằng số số học không kiểu có thể mang giá trị chính xác cho tới khi ngữ cảnh buộc nó vào kiểu; đó là lý do `const secondsPerMinute = 60` dùng được linh hoạt. Còn biến có kiểu thì mọi phép gán và conversion phải tuân luật kiểu của nó.

```
const kib = 1024
var bytes int64 = 8 * kib

workers := 3
perWorker := bytes / int64(workers) // conversion là chủ đích
```

Conversion không phải validation. `int64(3.9)` cắt phần lẻ, và ép một int lớn về kiểu nhỏ hơn có thể mất dữ liệu. Nếu dữ liệu đến từ config hoặc request, hãy kiểm tra range trước khi conversion. Đó là chỗ ta bảo vệ boundary, thay vì hy vọng loại dữ liệu ở phía ngoài sẽ luôn tử tế.

Scope và shadowing: hai biến cùng tên không phải một

Biến sống trong scope được bao bởi block. Điều này giúp giữ trạng thái cục bộ, nhưng `:=` có thể tạo biến mới và che biến cũ. Đây là lỗi rất hay gặp khi một nhánh xử lý error rồi vô tình trả biến bên ngoài chưa được cập nhật.

```
result := "unknown"
if ok := true; ok {
    result := "healthy" // biến mới, chỉ sống trong if
    _ = result
}
fmt.Println(result) // unknown
```

Khởi tạo ngắn trước điều kiện `if` là hợp lệ và thường hữu ích với `err`, vì nó giới hạn vòng đời biến. Nhưng nếu anh cần giá trị sau block, đừng dùng `:=` một cách phản xạ. Viết `result = "healthy"`, hoặc chọn tên khác, làm ý định rõ hơn và giảm một bug mà compiler không thể luôn đoán giúp.

if, switch và for là ít hơn để kiểm tra nhiều hơn

Go chỉ có `for` cho lặp. Nó có thể giống `while`, vòng ba phần cổ điển, hoặc lặp vô hạn có điều kiện thoát. Không có `while` riêng nghĩa là người đọc chỉ cần nhận diện một cấu trúc, không phải ba tên gọi cho ba biến thể gần nhau.

```
for attempts := 0; attempts < 3; attempts++ {  
    if attempts == 2 {  
        fmt.Println("last planned attempt")  
    }  
}
```

switch trong Go dừng sau case khớp; nó không fall through ngầm như C. Điều này làm case an toàn hơn để mô tả trạng thái. switch không có biểu thức cũng hữu ích để thay một chuỗi điều kiện đọc từ trên xuống, nhất là khi điều kiện liên quan range.

```
switch {  
case latencyMS < 0:  
    status = "invalid"  
case latencyMS < 200:  
    status = "healthy"  
case latencyMS < 1000:  
    status = "degraded"  
default:  
    status = "unhealthy"  
}
```

Thứ tự case là một phần logic. Nếu để latencyMS < 1000 lên trước, case dưới 200 sẽ không bao giờ chạy. Với threshold, bảng test tốt hơn cảm giác nhìn có vẻ đúng: nó ghi lại chính xác 199, 200, 999 và 1000 phải thuộc nhóm nào.

Zero value cần ngửi cảnh

Zero value giúp code nhỏ khởi động nhanh, nhưng nó không nói liệu dữ liệu có được cấu hình hay không. var port int là 0; trong một số API network, port 0 có thể yêu cầu hệ điều hành chọn port tạm. Nếu config của anh dùng 0 để biểu thị không điền, cần kiểm tra nó ở boundary chứ không để ý nghĩa mơ hồ trôi sâu vào chương trình.

Ta sẽ học slice, map và nil chi tiết hơn ở chương dữ liệu. Hiện tại chỉ cần nhớ: zero value là giá trị thật theo type, không phải một nhãn chung cho chưa có gì.

Lab: phân loại latency có ranh giới rõ

Lab labs/ch03-status có hàm ClassifyLatency. Đọc test trước, tự dự đoán kết quả ở các biên 0, 199, 200, 999 và 1000, rồi mới chạy test. Khi sửa threshold, anh phải sửa cả implementation lẫn expectation có chủ đích; đừng chỉ làm test xanh bằng cách nói điều kiện mơ hồ.

Bài tập

Hàm sau trả degraded cho latencyMS bằng 200 hay healthy? Không chạy trước. Sau đó đổi thứ tự hai case đầu và giải thích vì sao test boundary sẽ phát hiện lỗi.

```
func ClassifyLatency(latencyMS int) string {
    switch {
    case latencyMS < 200:
        return "healthy"
    case latencyMS < 1000:
        return "degraded"
    default:
        return "unhealthy"
    }
}
```

ĐÁP ÁN - chỉ đọc sau khi đã tự làm

200 không thỏa điều kiện nhỏ hơn 200, nên đi vào case thứ hai và trả degraded. Nếu đưa điều kiện nhỏ hơn 1000 lên đầu, mọi giá trị nhỏ hơn 200 cũng bị nó bắt trước; case healthy trở thành unreachable về mặt logic dù compiler không nhất thiết báo lỗi. Test với 199 sẽ thất bại, cho ta bằng chứng về rule bị đổi chứ không chỉ một output ngẫu nhiên.

Tóm lại

Kiểu và conversion làm dữ liệu có hình dạng rõ; tên biến và validation thêm ý nghĩa mà type đơn lẻ không mang nổi. if, switch và for ít nhưng đủ, miễn anh coi thứ tự điều kiện là logic có thể kiểm thử. Chương sau sẽ dùng hàm để đặt ranh giới nhỏ cho logic đó, rồi xử lý error mà không đẩy mọi tình huống xấu vào panic.

Chương 4. Hàm, error và defer: đặt ranh giới trước khi có sự cố

Một hàm tốt không chỉ gom code cho gọn. Nó đặt tên cho một quyết định, thu nhỏ số trạng thái phải theo dõi, và tạo nơi có thể kiểm thử mà không cần khởi động cả chương trình. Với tool DevOps/SRE, ranh giới này đặc biệt quan trọng: hàm parse config không nên âm thầm gọi network; hàm gọi network không nên giấu timeout; hàm main không nên chứa mọi policy lẫn I/O.

Parameter và result là hợp đồng nhỏ

Go cho phép nhiều result vì thao tác thực tế thường trả giá trị cùng trạng thái thất bại. Đây không phải ngoại lệ ngôn ngữ kiểu ném rồi bắt; error là một interface value được trả về như dữ liệu bình thường.

```
func ParsePort(raw string) (int, error) {
    port, err := strconv.Atoi(raw)
    if err != nil {
        return 0, fmt.Errorf("parse port %q: %w", raw, err)
    }
    if port < 1 || port > 65535 {
        return 0, fmt.Errorf("port %d is outside 1..65535", port)
    }
    return port, nil
}
```

0, err ở các nhánh lỗi không khẳng định 0 là một port hợp lệ. Nó chỉ là result không dùng được khi err khác nil. Caller phải xử lý error trước khi dùng result. Lời hứa này là convention mạnh của Go, không phải phép kiểm tra tự động của compiler; vì thế cách trả và kiểm tra error cần nhất quán.

Định dạng percent-w trong fmt.Errorf giữ error gốc để sau này code gọi có thể dùng errors.Is hoặc errors.As. Ta sẽ đi sâu thiết kế error ở chương abstraction. Ở đây, hãy giữ nguyên tắc gần hơn: thêm ngữ cảnh tại boundary có ích, nhưng đừng wrap cùng một lỗi ba lần chỉ để log trông dài hơn.

Đừng nuốt error

Giả sử startServer đã được khai báo hợp lệ, đoạn sau vẫn compile nhưng đánh mất bằng chứng quan trọng nhất khi config hỏng:

```
port, _ := strconv.Atoi(raw)
startServer(port)
```

Nếu raw là eighty, port nhận 0 và lỗi gốc biến mất. Hậu quả có thể là server nghe nhầm port hoặc thất bại ở nơi xa nguyên nhân. Dùng dấu gạch dưới chỉ khi anh đã chứng minh error không thể xảy ra hoặc thật sự không liên quan, và comment lý do nếu người đọc kế tiếp có thể nghi ngờ. Trong application code, điều đó hiếm hơn ta tưởng.

Variadic và named result: tiện, không phải mảnh khé

Parameter variadic như func Join(parts ...string) string nhận zero hoặc nhiều argument. Nó hợp với API có nghĩa một danh sách cùng loại, nhưng không thay thế một struct config khi tham số có vai trò khác nhau. Named result có thể làm chữ ký dễ đọc, song dùng return trần trong hàm dài khiến đường dữ liệu khó theo. Edition này ưu tiên return value, err rõ ràng trừ khi named result thực sự diễn đạt domain tốt hơn.

defer đăng ký cleanup ngay khi có resource

defer đặt một lời gọi để chạy khi hàm bao quanh return. Đối số của lời gọi defer được đánh giá ngay lúc gặp defer; bản thân lời gọi chạy sau. Điều này làm cleanup gần nơi acquire resource, nên đường return sớm không bỏ quên nó.

```
file, err := os.Open(path)
if err != nil {
    return err
}
defer file.Close()

return process(file)
```

Nhiều defer chạy theo LIFO: cái đăng ký sau chạy trước. Điều này thường đúng với resource lồng nhau. Nhưng defer không tạo retry, không log error Close một cách thần kỳ, và không làm một hàm chạy mãi có cleanup sớm hơn. Với vòng lặp lớn, đặt defer trong cùng một hàm có thể giữ resource tới cuối hàm; hãy tách thân mỗi iteration vào hàm nhỏ nếu cần release sớm.

panic và recover không phải đường error thông thường

panic dừng flow bình thường và unwinding stack, chạy các deferred call trên đường đi. Nó hợp với bất biến bị phá vỡ hoặc lỗi không thể tiếp tục an toàn, không phải với file không tồn tại, HTTP 500 hay input người dùng. Những tình huống ấy là expected failure và nên đi qua error.

recover chỉ bắt được panic khi gọi trực tiếp từ deferred function trong cùng goroutine. Nó không phải hàng rào toàn cục cho mọi goroutine, và dùng nó để che bug sẽ khiến process trông còn sống nhưng state đã không đáng tin. Ở HTTP server production, một recovery boundary hẹp có thể giúp cô lập request; nó vẫn phải ghi log, tăng metric và trả lỗi có kiểm soát. Ta sẽ quay lại khi đã có HTTP và observability.

Lab: parse port, giữ nguyên nguyên nhân

labs/ch04-portcheck parse string thành TCP port. Test table-driven bao phủ input hợp lệ, không phải số, 0 và 65536. Đọc từng test trước; sau đó thử bỏ range check và quan sát chính xác test nào báo rule bị mất. Đừng đổi expected chỉ để test xanh: expected là policy của tool, không phải vật cản.

Bài tập

Đây là snippet cố ý chỉ tập trung vào error path, không phải file hoàn chỉnh. Hãy chỉ ra hai lỗi thiết kế trong nó và viết lại ý tưởng, chưa cần code:

```
func Run(rawPort string) {  
    port, _ := strconv.Atoi(rawPort)  
    defer fmt.Println("done")  
    panic(startServer(port))  
}
```

ĐÁP ÁN - chỉ đọc sau khi đã tự làm

Thứ nhất, error từ parse bị nuốt, nên input xấu bị biến thành port 0 mà không còn bối cảnh. Thứ hai, panic được dùng như đường điều khiển cho lỗi có thể dự đoán từ startServer; caller không còn lựa chọn xử lý hay trình bày lỗi. Hàm nên trả error, kiểm tra parse và range trước, rồi để main quyết định log và exit code. defer có thể giữ cho cleanup hoặc log thật sự cần chạy khi return, chứ không bù cho policy lỗi sai.

Tóm lại

Hàm là ranh giới cho logic và test. Multiple result đặt value cạnh error; xử lý error gần nơi nó xảy ra giữ nguyên bằng chứng. defer là công cụ cleanup có thứ tự, còn panic và recover là cơ chế hẹp cho trạng thái bất thường, không phải shortcut để khởi thiết kế error path. Chương dữ liệu sẽ dùng những ranh giới này để xem slice, map và pointer chia sẻ hay sao chép điều gì.